

Security-Aware Adaptive Computation Offloading in Mobile Edge Computing Using Reinforcement Learning and Deep Q-Networks

BRINDESHWAR SHARMA¹

¹Department of Information Technology, Manipal University Jaipur, Jaipur, India (e-mail: brinde.sharma@gmail.com)

Corresponding author: Brindeshwar Sharma (e-mail: brinde.sharma@gmail.com).

ABSTRACT Mobile Edge Computing (MEC) enables resource-constrained mobile devices to offload computation-intensive tasks to nearby edge servers. Existing computation offloading approaches primarily optimise latency, energy consumption, or resource allocation, but often do not consider security constraints and multi-user queue stability within a unified decision framework. This paper studies, in simulation, a security-aware adaptive offloading framework that combines an analytical bandwidth threshold, workload classification, reinforcement learning, Lyapunov drift-plus-penalty queue control, Object Dependency Graph (ODG)-based vulnerability scoring, and a Deep Q-Network (DQN) over a continuous state. The framework derives a break-even bandwidth of 13.71 Mbps for time-beneficial offloading. A drift-plus-penalty admission controller bounds the edge queue; in a deterministic illustration it holds the queue at 21 tasks against 297 for greedy admission. ODG gating keeps labelled sensitive objects on the device, although a ratio-based score is shown to let transitively dependent objects leak. Over 10 random seeds, a DQN behind hard gates reaches a reward of -1.91 per step, against -2.07 for local execution and -1.97 for a rule-based pipeline, while a Q-table at the same discount factor falls to -2.42. The study is simulation-only, uses N-Queens as a workload proxy, assumes a single edge server and a reward derived from its own cost model, and these limitations are discussed explicitly.

INDEX TERMS Computation offloading, deep Q-network, Lyapunov optimization, mobile edge computing, object dependency graph, reinforcement learning, security-aware offloading.

I. INTRODUCTION

MOBILE applications are becoming increasingly computation-intensive due to the rapid growth of artificial intelligence, augmented reality, healthcare monitoring, autonomous systems, and Internet of Things (IoT) services [1], [2]. Although modern smartphones and edge devices continue to improve in processing capability, they remain constrained by battery capacity, computation resources, thermal limitations, and storage availability. Executing complex workloads entirely on mobile devices can therefore lead to excessive energy consumption, increased latency, and degraded user experience [3], [4].

Mobile Edge Computing (MEC) has emerged as an effective paradigm for addressing these limitations by enabling mobile devices to offload computationally intensive tasks to nearby edge servers [2], [5]. Unlike traditional cloud computing, MEC places computational resources closer to end users, thereby reducing communication latency and improving re-

sponsiveness. Numerous computation offloading strategies have been proposed to determine whether a task should be executed locally on the mobile device or remotely on an edge server. The existing approaches are mainly aimed at reducing latency, energy consumption, maximizing throughput, or optimizing the utilization of resources [1], [6], [7].

Although great progress has been made in MEC research, there are still some problems that have not yet been solved. First, many of the offloading frameworks make a number of assumptions about network conditions that are stable and do not adapt dynamically to varying bandwidth availability [3], [4]. Second, conventional approaches often ignore queue congestion at edge servers, which can significantly increase processing delays under heavy workloads [8], [9]. Third, existing offloading approaches focus on performance metrics and ignore security and privacy issues of sensitive data offloading to external infrastructures [10], [11]. Finally, traditional Q-Learning-based offloading methods rely on dis-

crete lookup tables that struggle to scale when the state space becomes continuous and high-dimensional [12]–[14].

To address these limitations, this paper studies a security-aware adaptive computation offloading framework that integrates analytical modelling, reinforcement learning, queue stability control, vulnerability-aware decision making, and deep reinforcement learning. The framework first derives an analytical break-even bandwidth threshold that determines when offloading becomes beneficial. Computational workloads are characterised with the N-Queens benchmark problem to distinguish lightweight and heavyweight tasks. A Q-Learning agent learns adaptive offloading policies from battery level and workload characteristics, while a Lyapunov drift-plus-penalty controller maintains queue stability in multi-user edge environments.

The framework also includes a security-aware decision layer that is implemented using Object Dependency Graph (ODG) analysis. Application objects receive vulnerability scores based on the access they have to sensitive data such as user identity, medical records, location data, and biometric data. Highly sensitive items are not offloaded, and hence privacy risks and information leakage are lowered. The framework further extends conventional Q-Learning by incorporating a Deep Q-Network (DQN) that can be applied to continuous state spaces, such as battery level, task complexity, vulnerability score, available bandwidth and server load.

This work can be summarized as follows:

- An analytical bandwidth threshold model for determining the conditions under which edge offloading is beneficial.
- A workload-aware computation offloading framework that uses reinforcement learning for adaptive decision making.
- A Lyapunov drift-plus-penalty admission controller for the edge queue in multi-user MEC environments, with a simple queue-length bound.
- ODG-based vulnerability scoring for security-aware offloading decisions, together with a measurement of how often such a gate leaks on random dependency graphs.
- An extension of tabular Q-Learning to a Deep Q-Network that handles a continuous state, compared against tabular Q-Learning and rule-based baselines over multiple random seeds.

All results come from a simulator; the code, raw outputs and the scripts that generate every figure and table in this paper are available in the accompanying repository.¹

The remainder of this paper is organized as follows. Section II reviews existing literature on MEC offloading and reinforcement learning approaches. Section III presents the system model and optimization objective. Section IV describes the proposed adaptive offloading algorithm. Section V discusses the experimental configuration. Sections VI through X describe workload classification, queue stability, security-aware offloading, and the DQN extension. Sec-

tion XI presents the results and discussion, followed by complexity analysis, limitations, conclusion, and future scope.

II. RELATED WORK

Computation offloading has received extensive research in both the Mobile Cloud Computing and the Mobile Edge Computing settings. Early work such as CloneCloud, MAUI and MACS showed that mobile applications could be partitioned and their execution moved to remote servers to save energy and reduce execution time [3], [4], [15]. While these early frameworks laid the groundwork for the offloading paradigm, they did not have adaptive intelligence, multi-user queue control or any type of security-aware decision making.

A. ANALYTICAL OFFLOADING AND THRESHOLD-BASED OFFLOADING

Lin *et al.* survey computation offloading toward edge computing, including wireless-channel and execution-time based criteria for binary offloading decisions [5]. Kovachev and Klamka applied the N-Queens benchmark to the MACS framework to measure the offloading benefit for different task sizes, showing that costly computations benefit significantly from remote execution, while small tasks are dominated by network overhead [15]. These analytical methods offer clear decision criteria, but are not able to account for the changing state of the battery, changing workload, or sensitivity of the data being offloaded.

B. REINFORCEMENT LEARNING FOR ADAPTIVE OFFLOADING

Kiran *et al.* introduced a Software Defined Edge Cloudlet framework with the aid of Q-Learning for joint resource allocation and computation offloading, showing that reinforcement learning can outperform greedy and random strategies in minimizing delay [12]. Huang *et al.* further built on this research with a deep reinforcement learning algorithm for joint task offloading and bandwidth allocation in multi-user MEC systems that demonstrated higher performance under different channel conditions [14]. Tang and Wong presented a model-free distributed algorithm based on deep reinforcement learning that allows each device to make offloading decisions without knowing the state of other devices, or the task models [13]. Later, He *et al.* investigated multi-agent deep reinforcement learning for dynamic task offloading in D2D-MEC networks to reduce the average task delay while meeting the deadline requirements of distributed decision makers [16].

C. QUEUE STABILITY AND CONGESTION CONTROL

Bi *et al.* proposed a deep reinforcement learning framework with Lyapunov guidance (LyDROO) to solve the online computation offloading problem in MEC networks with long-term queue and power constraints, which significantly outperforms the greedy baseline in multi-user MEC networks [17]. Bae *et al.* also theoretically linked reinforcement learning to Lyapunov optimization, establishing a link between stability

¹github.com/brindeshwar/security-aware-mec-offloading

guarantees and learned policies [18]. In this direction, Chen *et al.* directly addressed multi-user contention for edge resources and proposed efficient decentralized offloading game formulations for mobile-edge cloud computing [9]. Wang *et al.* optimized the energy consumption and latency of offloading and computing in wireless powered MEC systems with energy constraints while considering multi-user resource sharing, where they showed energy-latency tradeoffs [7].

D. SECURITY AND PRIVACY IN OFFLOADING DECISIONS

Although performance improvements have been made, few offloading frameworks have taken security into account. Zhu *et al.* presented Object Dependency Graphs for vulnerability assessment of mobile applications offloaded to cloud infrastructure, which identifies the sensitive objects and their inter-dependencies [10]. Their method models application-level privacy risk in the offloading context, but was static and not used with any reinforcement learning or queue stability mechanism. Gan *et al.* took privacy concerns a step further by introducing differentially private deep Q-learning for pattern privacy preservation in MEC offloading, where they demonstrated that privacy guarantees can be incorporated into the learning objective [11]. Zhao *et al.* used deep Q-networks to solve the task offloading problem in a cooperative intrusion detection system and showed that the security-functional workload can be scheduled at the edge [19].

E. CONTINUOUS-STATE LEARNING AND DEEP Q-NETWORKS

Zhang *et al.* proposed an optimization of the energy-latency trade-off in MEC under different network conditions based on analytical energy models [6]. Mao *et al.* gave a comprehensive study from the communication perspective, setting the system models and optimization goals for many of the following DRL-based offloading studies [2]. Yang *et al.* used DQN-based task offloading for multimedia data analysis in edge-computing-enabled Internet of Vehicles (IoV) to support decision making in multi-variety task environments [20]. Anand and Karthikeyan proposed an Adaptive and Intelligent Customized DQN (AICDQN) for priority-aware, energy-efficient task scheduling in MEC, demonstrating that customized DQN formulations can yield better performance than standard ones on dynamic IoT workloads [21].

F. DEPENDENCY-AWARE AND GRAPH-BASED OFFLOADING

Recent efforts have turned to explicitly modelling task inter-dependencies. Liu *et al.* introduced an online task offloading strategy for IoV with DRL that optimizes task latency and energy consumption based on cooperative base stations and edge servers and considers the task dependency graph [22]. A similar approach was proposed for dependency-aware dynamic offloading in multi-device MEC, where the execution of local tasks is adjusted according to the status of edge nodes and the precedence of the tasks under time-varying fading channels (the DODA-DT algorithm) [23]. The DCEDRL

method extended this work by introducing density clustering and ensemble learning with DRL for the offloading decision, which uses multiple neural networks trained together to alleviate the sensitivity to dynamic channel and load changes [24]. A parallel approach with asynchronous training and parallel exploration was shown to improve convergence stability for multi-user offloading decisions in dynamic edge environments [25].

G. RECENT ADVANCES: TEMPORAL, COOPERATIVE AND INTELLIGENT FRAMEWORKS

Zhang *et al.* discussed that time-ordered task graphs need special scheduling policies other than the standard Markov-state formulations [26]. For large-scale dynamic workloads, Sun *et al.* proposed a three-layer cloud-edge-device task offloading model based on DRL, which has advantages over two-layer architectures [27]. Jalal *et al.* proposed Public Edge as a Service (PEaaS) as an intermediate layer and designed a PPO-based scheduler, which includes mobility, congestion, cost and energy in a single reward function for intelligent regional offloading [28]. Dependent-task offloading with DRL has also been studied for general edge computing [29]. Table 1 summarizes the capabilities of existing MEC offloading frameworks against the framework studied here across seven dimensions.

H. SUMMARY OF GAPS

It has been found from the literature that current computation offloading frameworks consider only a subset of the decision factors necessary for real-world MEC deployment. Previous works have individually investigated bandwidth-aware offloading, reinforcement learning, Lyapunov-based queue control, security-aware computation, and deep reinforcement learning; however, few studies have combined these techniques into a unified end-to-end decision-making framework. Motivated by this gap, we study an adaptive offloading framework that integrates bandwidth thresholding, workload-aware learning, queue stability control, ODG-based vulnerability analysis, and DQN-based continuous-state optimization, and we evaluate it against explicit baselines.

III. SYSTEM MODEL

The proposed system consists of a mobile device, an edge server, a wireless communication link, and a decision engine. Each incoming task is evaluated using network conditions, task complexity, battery level, vulnerability score, and server load.

A. LOCAL EXECUTION MODEL

For a task requiring n CPU cycles and a local device frequency f_{local} , the local execution time is

$$T_{\text{local}} = \frac{n}{f_{\text{local}}}. \quad (1)$$

TABLE 1. Qualitative comparison of MEC offloading frameworks. Each cell reflects the focus stated in the cited work, as read by the author, and is not a quantitative benchmark. “Yes” means the dimension is addressed, “Partial” that it is addressed only in part, and “Static” that security is assessed offline only. The last row describes the framework studied in this paper, in simulation.

Framework	Latency opt.	Energy opt.	AI-driven	Queue stability	Security-aware	Continuous state	Dependency-aware
CloneCloud, MAUI, MACS [3], [4], [15]	Yes	Yes	No	No	No	No	No
Lin <i>et al.</i> [5]	Yes	No	No	No	No	No	No
Barbarossa <i>et al.</i> , Bi <i>et al.</i> [8], [17]	Yes	Yes	Partial	Yes	No	No	No
Kiran <i>et al.</i> , Huang <i>et al.</i> [12], [14]	Yes	Yes	Yes	No	No	No	No
Tang & Wong, He <i>et al.</i> [13], [16]	Yes	Yes	Yes	No	No	Yes	No
Zhu <i>et al.</i> , Zhao <i>et al.</i> [10], [19]	No	No	Partial	No	Static	No	No
Gan <i>et al.</i> [11]	Yes	Yes	Yes	No	Yes	No	No
Yang <i>et al.</i> , Anand & Karthikeyan [20], [21]	Yes	Yes	Yes	No	No	Yes	No
Liu <i>et al.</i> , Fang <i>et al.</i> , Wang <i>et al.</i> [22], [23], [29]	Yes	Yes	Yes	No	No	Yes	Yes
This work (simulation)	Yes	Yes	Yes	Yes	Yes	Yes	Yes

B. EDGE OFFLOADING MODEL

For a task with input data size m bits, transmission rate R , and edge-server frequency f_{remote} , the offloading time is

$$T_{\text{offload}} = \frac{m}{R} + \frac{n}{f_{\text{remote}}} (+T_{\text{setup}} + T_{\text{queue}}). \quad (2)$$

The terms in parentheses are the connection setup delay and the waiting time at a loaded edge server; they are zero in the analytical derivation and non-zero in the simulation (Section V). Offloading is beneficial when $T_{\text{offload}} < T_{\text{local}}$. Solving for the break-even bandwidth gives

$$R^* = \frac{m}{\frac{n}{f_{\text{local}}} - \frac{n}{f_{\text{remote}}}}. \quad (3)$$

Using $n = 10^9$ cycles, $m = 8$ Mb, $f_{\text{local}} = 1.2$ GHz, and $f_{\text{remote}} = 4.0$ GHz, the break-even bandwidth is $R^* = 13.71$ Mbps. Including the 50 ms setup delay moves the break-even point to 15.0 Mbps. This threshold acts as the first decision gate: if the available bandwidth is below R^* , the task is executed locally. Fig. 1(a) shows the two completion-time curves and their crossing points for the reference task. Fig. 1(b) shows how R^* moves with the edge speed-up and the setup delay: a faster edge lowers the threshold, while a longer setup delay raises it sharply as the speed-up approaches one.

C. ENERGY MODEL

Local energy consumption is modelled as

$$E_{\text{local}} = P_{\text{cpu}} T_{\text{local}}, \quad (4)$$

and the device-side energy of offloading as

$$E_{\text{offload}} = P_{\text{tx}} T_{\text{tx}} + P_{\text{idle}} T_{\text{wait}}, \quad (5)$$

where P_{tx} is the transmission power, $T_{\text{tx}} = m/R$, and P_{idle} is the idle power while waiting for the remote execution (T_{wait} covers setup, edge computation and queueing). The simulation uses $P_{\text{cpu}} = 0.9$ W, $P_{\text{tx}} = 1.3$ W and $P_{\text{idle}} = 0.15$ W; these are assumed values, not measurements.

D. OPTIMIZATION OBJECTIVE

The framework jointly considers latency, energy consumption, queue congestion, and security. Latency, energy and security enter the per-task reward that the learning agents maximise,

$$r = -(T + \beta(b)E) - a\sigma(V), \quad (6)$$

where $a \in \{0, 1\}$ is the decision (1 = offload), $b \in [0, 1]$ is the battery level, $\beta(b) = 0.5(1 + 3(1 - b))$ weights energy more heavily as the battery drains, and $\sigma(V)$ is a penalty for offloading a task with vulnerability score V : $\sigma = 0$ for $V < 0.33$, 1.5 for $0.33 \leq V < 0.66$ and 3.0 for $V \geq 0.66$. Queue congestion enters through the edge load, which raises T_{queue} for later tasks, and through the explicit admission controller of Section VII. When the battery reaches zero the episode ends with an extra penalty of 5.

IV. PROPOSED ALGORITHM

The proposed algorithm performs a sequence of decision checks before a task is either executed locally or offloaded to the edge server. The decision process is designed to ensure that offloading occurs only when it is beneficial, secure, and stable from the perspective of the edge infrastructure.

- 1) Receive the incoming task request.
- 2) Measure the available bandwidth.
- 3) Compare the bandwidth with the break-even threshold; if it is lower, execute locally.
- 4) Compute the ODG vulnerability score of the task's objects.
- 5) If the score is at or above the gate τ_V , force local execution.
- 6) Otherwise evaluate the learned policy (Q-Learning or DQN), whose input includes the task complexity, the battery level and the edge load.
- 7) Check the edge load; if it is at or above the load limit Λ , execute locally.
- 8) Otherwise offload the task to the edge server.

Fig. 2 shows the gates in order. The workload classification of Section VI characterises how task size affects the offloading

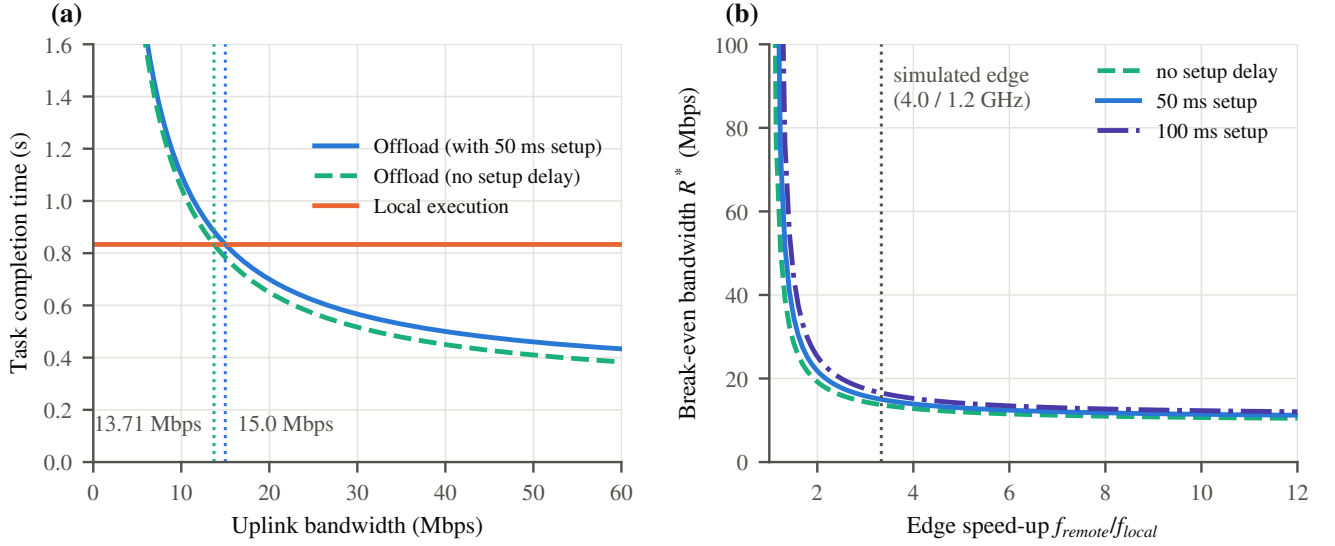


FIGURE 1. Analytical break-even bandwidth. (a) Completion time of the reference task (1 Gcycle, 8 Mb) versus uplink bandwidth: local execution (orange, constant), offloading without setup delay (green, dashed) and with a 50 ms setup delay (blue). The curves cross at 13.71 and 15.0 Mbps (dotted lines). (b) Break-even bandwidth R^* as a function of the edge-to-device frequency ratio for three setup delays; the dotted line marks the simulated ratio 4.0/1.2.

benefit; in the implemented pipeline it is carried by the complexity input of the learned policy. The multi-user admission controller of Section VII regulates how many offloaded tasks reach the edge queue per slot, and is evaluated separately.

Algorithm 1: Offloading decision for one task.

- 1: **Input:** bandwidth R , vulnerability V , battery b , complexity c , edge load ℓ
- 2: **if** $R < R^*$ **or** $V \geq \tau_V$ **or** $\ell \geq \Lambda$ **then**
- 3: **return** local
- 4: **end if**
- 5: $a \leftarrow \arg \max_{a'} Q(s, a')$ {Q-table or DQN}
- 6: **return** offload if $a = 1$, otherwise local

V. EXPERIMENTAL SETUP

A. SIMULATION ENVIRONMENT

The framework was implemented in Python using NumPy and Matplotlib, including the DQN, whose forward and backward passes are written by hand. All experiments run on a single workstation; the full set of experiments takes roughly twenty minutes. The MEC environment consists of a mobile device operating at 1.2 GHz and an edge server operating at 4.0 GHz. A task of complexity $c \in (0, 1]$ requires $n = c \cdot 2 \times 10^9$ cycles and $m = c \cdot 16$ Mb of input data, so that $c = 0.5$ is the reference task (one billion cycles, 8 Mb); because n and m scale together, the break-even bandwidth of Eq. (3) does not depend on c . A fixed network setup delay of 50 ms represents connection initialization overhead. The bandwidth follows a log-random walk between 1 and 100 Mbps. The edge load $\ell \in [0, 1]$ rises with offloaded work and decays otherwise, and adds a queueing delay of $0.8\ell^2$ seconds, so that offloading has a cost for later tasks. Task complexity c is drawn uniformly from $(0.05, 1]$ and the vulnerability score V from a Beta(1, 2) distribution (most tasks are low-risk; about 12% have $V \geq 0.66$). An episode ends after 60 tasks or when the battery reaches zero.

TABLE 2. Configuration of the simulation and of the learning agents. Power and battery values are assumptions chosen to be plausible, not measurements.

Parameter	Value
Device / edge CPU	1.2 GHz / 4.0 GHz
Setup delay, bandwidth range	50 ms, 1–100 Mbps
Power: CPU / transmit / idle	0.9 W / 1.3 W / 0.15 W
Battery budget	60 J
Security penalty σ (MED / HIGH)	1.5 / 3.0
ODG gate τ_V , load limit Λ	0.4, 0.85
Discount factor γ	0.95
Q-table learning rate	0.1
DQN learning rate (Adam)	0.001
DQN replay buffer / batch	20,000 / 64
DQN target-network update	every 200 steps
Exploration ϵ	1.0 to 0.05, linear over first 60% of episodes
Training episodes, seeds	1500, 10
Queue: arrival / capacity	Poisson(10) / 7 tasks per slot

B. LEARNING CONFIGURATION

Table 2 lists the configuration. The Q-table sees the same observation as the DQN after discretisation (5 battery levels, 4 complexity levels, 3 vulnerability buckets, 6 bandwidth bins and 4 load levels, 1,440 states). The DQN is a $5 \rightarrow 64 \rightarrow 64 \rightarrow 2$ ReLU network with 4,674 weights, experience replay, a target network and the Adam optimizer. Both agents are trained for 1500 episodes on each of 10 random seeds and are evaluated greedily on 200 fresh episodes per seed, generated from a random stream that is shared by all policies of the same seed.

C. BASELINES AND METRICS

Every learned policy is compared with simple policies: *always local*, *always offload* (greedy), *random*, *threshold only*

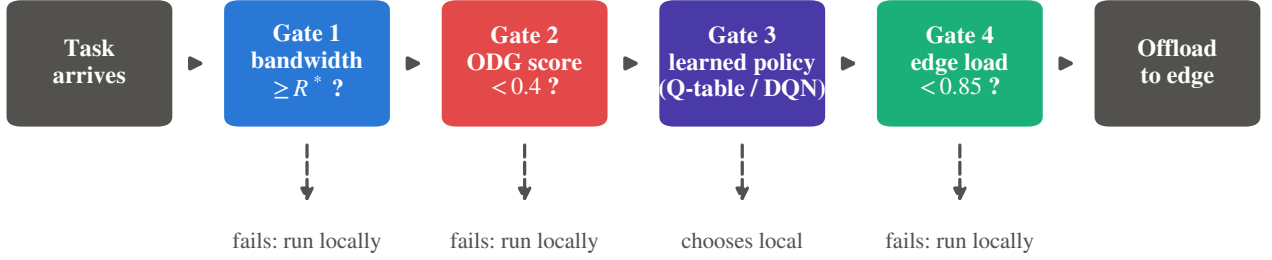


FIGURE 2. The decision pipeline as implemented. A task is offloaded only if it passes the bandwidth gate ($R \geq R^*$), the ODG gate (score below $\tau_V = 0.4$), the learned policy, and the edge-load gate (load below ~ 0.85); failing any gate, or a “local” decision by the learned policy, executes the task on the device.

(offload iff $R \geq R^*$), *threshold plus ODG gate*, and a *rule pipeline* that adds the load gate. A *myopic oracle*, which picks the action with the higher immediate reward using the true model, serves as a reference that is not realisable without knowing the model. We report the reward per step, the latency and energy per task, the share of offloaded tasks, the share of high-vulnerability tasks ($V \geq 0.66$) that were offloaded, and the share of episodes that deplete the battery, each as a mean over seeds with a 95% confidence interval.

VI. WORKLOAD CLASSIFICATION AND Q-LEARNING

The first intelligent decision layer of the framework is based on workload characterisation and reinforcement learning. Since lightweight and heavyweight computational tasks exhibit significantly different execution characteristics, a workload-aware policy is required before deciding whether a task should be executed locally or offloaded [12], [14].

A. N-QUEENS WORKLOAD PROXY

To emulate computational workloads with varying complexity, the N-Queens problem was adopted as a CPU-bound proxy. It is used in computation offloading research because the computational cost increases rapidly with problem size while maintaining a consistent problem structure [15]. Here the task is to count all solutions of the N -Queens problem with a bitmask backtracking solver; the local time is measured on the machine running the simulation (median of three runs), and the edge time is *modelled*, not measured: the local time scaled by the CPU ratio $f_{\text{local}}/f_{\text{remote}}$, plus the 50 ms setup delay and the time to send 8 Mb at the given bandwidth. Tasks with small N are classified as lightweight, since the setup and transfer overhead exceeds the compute saving, whereas larger N are heavyweight. Fig. 3 shows that the modelled edge execution overtakes local execution at $N = 12$ on a 50 Mbps link and at $N = 13$ on a 10 Mbps link, so a slower link moves the crossover to heavier tasks. Because local times are measured on a desktop CPU rather than on a phone, the absolute values and the crossover N depend on the machine; the figure illustrates the shape of the trade-off rather than a

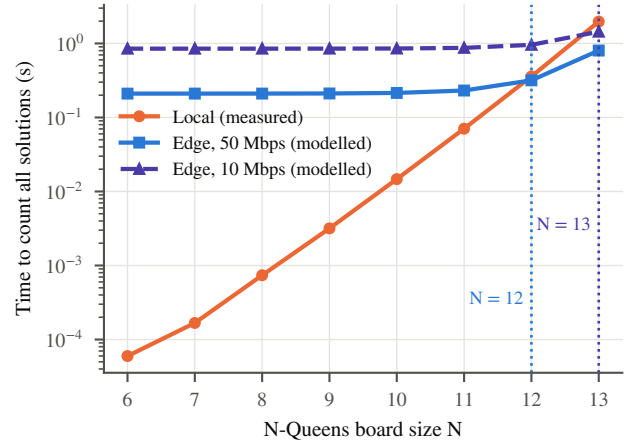


FIGURE 3. N-Queens workload proxy (log scale). Measured local time to count all solutions (orange) versus the modelled edge time at 50 Mbps (blue) and 10 Mbps (violet dashed). Dotted lines mark the first board size at which the edge becomes faster. Edge times are modelled from the local time, the CPU ratio, a 50 ms setup delay and the transfer of 8 Mb; they are not measured on a remote machine.

device measurement.

B. TABULAR Q-LEARNING AGENT

The tabular agent observes five quantities, discretised: battery level, task complexity, vulnerability bucket, bandwidth relative to the threshold, and edge load. The available actions are to execute locally or to offload to the edge server. The objective of the Q-Learning agent is to maximize the long-term cumulative reward [12]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (7)$$

where $Q(s, a)$ is the state-action value, α is the learning rate, γ is the discount factor, r is the immediate reward of Eq. (6) and s' is the next state. The reward trades latency and energy against the security penalty. The environment is dynamic: the battery drains with every task, the edge load increases when tasks are offloaded, and the bandwidth moves, so that

an offloading decision affects the later states. The agent learns by interacting with this environment and gradually modifying its policy [13], [14]. How quickly it learns, and how well, is examined in Section XI using the reward of the greedy policy on fresh episodes; unlike the sum of Q-values, this quantity directly shows whether the policy is improving.

VII. QUEUE STABILITY USING LYAPUNOV CONTROL

While reinforcement learning can detect beneficial offloading opportunities, aggressive offloading can cause edge servers to be overloaded, with resulting queue growth. To overcome this challenge, a Lyapunov drift-plus-penalty admission controller is added to the framework [17], [18].

A. CONTROLLER

Let $Q(t)$ be the length of the edge queue at time slot t , $a(t)$ the number of tasks admitted to the edge and μ the number of tasks served per slot, so that $Q(t+1) = \max\{Q(t) + a(t) - \mu, 0\}$. The Lyapunov function and drift are

$$L(Q(t)) = \frac{1}{2}Q(t)^2, \quad (8)$$

$$\Delta(Q(t)) = \mathbb{E}[L(Q(t+1)) - L(Q(t)) | Q(t)]. \quad (9)$$

To maintain queue stability the controller minimizes the drift-plus-penalty expression

$$\Delta(Q(t)) + V \cdot \text{Penalty}(t), \quad (10)$$

where V is a control parameter that balances queue stability against performance. Each arriving task i has an offloading benefit $b_i \in (0, b_{\max}]$ (for example, the latency it saves relative to local execution), so the penalty is $-\sum_{i \text{ admitted}} b_i$. Since $\Delta(Q) \leq B + Q(t) \mathbb{E}[a(t) - \mu | Q(t)]$ for a constant B , minimizing the bound in each slot gives a simple rule: admit task i if and only if

$$V b_i > Q(t), \quad (11)$$

and execute it locally otherwise.

Queue bound. With $b_i \leq b_{\max}$ and at most $A_{\max}$ admitted tasks per slot, $Q(t) \leq Vb_{\max} + A_{\max}$ for all t . Proof: if $Q(t) \geq Vb_{\max}$, rule (11) admits no task, so Q cannot increase in that slot; Q can therefore exceed $Vb_{\max}$ only by the tasks admitted in a single slot, at most $A_{\max}$. The parameter V trades queue length against the benefit gained: a larger V admits more tasks at the cost of a longer queue.

B. RESULTS

We evaluate the controller in two settings (Fig. 4 and Table 3). The first is the deterministic illustration used earlier in this study: 10 tasks arrive per slot, the server serves 7, and a simple throttle admits only 4 tasks per slot once the queue reaches 20. Over 100 slots, greedy admission grows by $10 - 7 = 3$ tasks per slot and ends at 297 tasks (3×99), whereas the throttled queue stays at a maximum of 21 and redirects 276 tasks to local execution (Fig. 4a). The value 297 is a consequence of the chosen rates and horizon; it illustrates that greedy admission is unbounded when arrivals exceed capacity, and

TABLE 3. Edge-queue control under stochastic arrivals (Poisson, mean 10 per slot; capacity 7 per slot; 2,000 slots; mean over 20 seeds). DPP is drift-plus-penalty. "Admitted" is the share of arriving tasks sent to the edge; "benefit" is the summed offloading benefit of admitted tasks per slot (mean $\pm$ 95% CI).

Policy	Mean Q	Max Q	Admitted (%)	Benefit/slot
Greedy	3024.5	6026	100	6.00 $\pm$ 0.02
Threshold $\Theta = 20$	17.6	32	70	4.20 $\pm$ 0.01
DPP, $V = 5$	2.3	15	66	4.20 $\pm$ 0.01
DPP, $V = 10$	4.3	16	70	4.71 $\pm$ 0.01
DPP, $V = 20$	8.7	19	70	4.92 $\pm$ 0.01
DPP, $V = 50$	21.9	35	70	5.00 $\pm$ 0.01
DPP, $V = 100$	43.8	61	70	5.03 $\pm$ 0.01

the boundedness of the throttled queue follows directly from its rule.

The second setting is stochastic: Poisson arrivals with mean 10 per slot, benefits $b_i \sim U(0.2, 1)$, service capacity 7 per slot, 2,000 slots and 20 seeds. Greedy admission reaches a mean queue of 3025 tasks, while the fixed threshold of 20 keeps a mean of 17.6 and a maximum of 32 because Poisson bursts overshoot it. The drift-plus-penalty rule with $V = 20$ holds the mean at 8.7 and the maximum at 19, within the bound $Vb_{\max} + A_{\max}$, while collecting a benefit of 4.92 per slot compared with 4.20 for the fixed threshold. Both controllers admit about 70% of arrivals (the service capacity), but drift-plus-penalty chooses the higher-benefit tasks. Increasing V raises both the mean queue and the benefit, with diminishing returns (Fig. 4c).

VIII. SECURITY-AWARE OFFLOADING

Most computation offloading frameworks consider only latency and energy efficiency. However, many mobile applications process highly sensitive information such as personal identifiers, healthcare records, location information, and biometric measurements. Offloading such data without considering the security implications can introduce significant privacy risks [1], [11].

To address this challenge, the framework introduces an Object Dependency Graph (ODG)-based vulnerability scoring mechanism. The ODG models application objects and their dependencies. Objects that directly access sensitive information are assigned the highest vulnerability score [10].

A. VULNERABILITY SCORE AND GATE

The vulnerability score of an object i is defined as the share of the objects reachable from it that are sensitive,

$$V_i = \frac{\text{sensitive objects reachable from } i}{\text{all objects reachable from } i}, \quad 0 \leq V_i \leq 1, \quad (12)$$

computed by depth-first search over transitive dependencies; a sensitive object has $V_i = 1$. $V_i = 0$ indicates that the object does not depend on sensitive data and can be offloaded, and $V_i = 1$ indicates a highly sensitive object. The vulnerability score is incorporated into the offloading decision in two ways. First, a hard gate: objects with $V_i \geq \tau_V = 0.4$ are

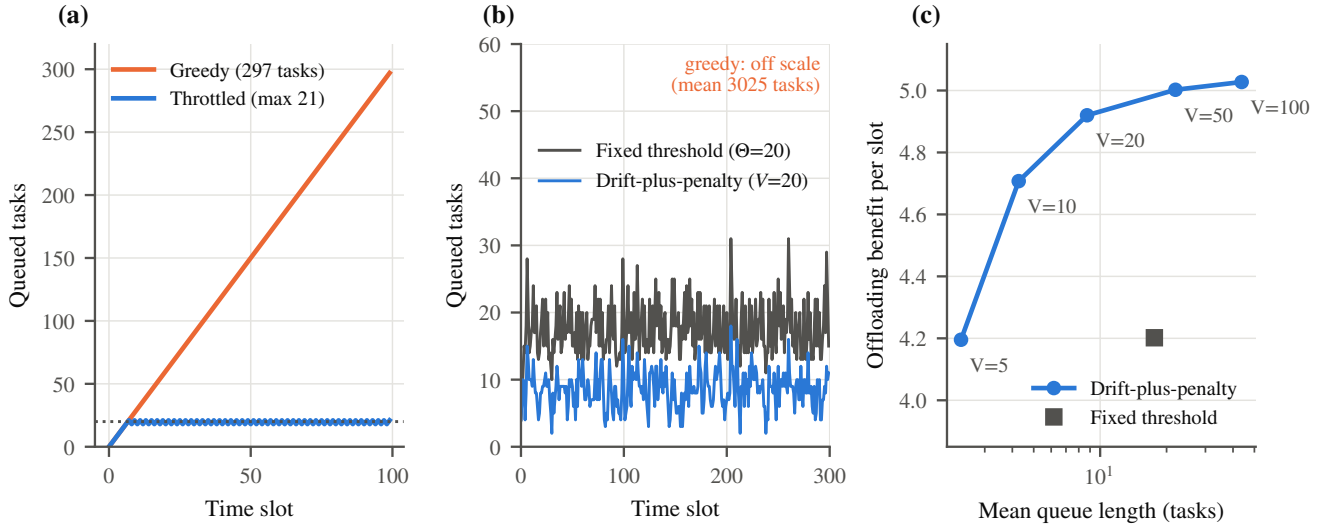


FIGURE 4. Edge-queue control. (a) Deterministic illustration: constant arrivals (10 per slot) exceed the capacity (7 per slot); greedy admission grows linearly to 297 tasks after 100 slots while the throttle keeps the queue at most 21 (dotted line: threshold 20). (b) Stochastic arrivals (Poisson, mean 10), first 300 slots of one seed: the fixed threshold overshoots in bursts, drift-plus-penalty with $V = 20$ keeps the queue shorter; greedy admission is off scale. (c) Trade-off between mean queue length (log axis) and offloading benefit per slot as V varies (mean over 20 seeds), with the fixed threshold for reference.

never offloaded, regardless of bandwidth availability or the learned policy. Second, the learned agents see V as an input and receive the security penalty $\sigma(V)$ of Eq. (6) with the thresholds Low ($V < 0.33$), Medium ($0.33 \leq V < 0.66$) and High ($V \geq 0.66$).

B. EXAMPLE APPLICATION

The example application is a hand-built healthcare/fitness app with twelve objects: four sensitive ones (user ID, heart rate, medical record, location), four safe utilities and four composite objects (Fig. 5a). All four sensitive objects score 1.0 and stay on the device; this follows from their labelling together with a hard rule, so it is a property of the gate rather than an empirical result. The composite objects *analyzeHealth* and *buildReport* score 0.50 and stay local, whereas *syncToCloud* (0.29) and *runAllInference* (0.33) fall below the gate and may be offloaded. Note that *syncToCloud* depends on *getUserID* through *buildReport*; the ratio of Eq. (12) dilutes this dependency because the object also reaches several safe ones.

C. A STRICTER TAINT GATE AND ITS COST

To quantify this effect we compare the ratio score with a conservative *taint* score that is 1 whenever an object can reach any sensitive object, and 0 otherwise. On 300 random dependency graphs (30 objects, 20% sensitive, edge probability 0.12), the paper's gate (ratio score, $\tau_V = 0.4$) allows 74% of the objects that reach sensitive data to offload while forcing only 11% of the non-sensitive objects to stay local. The taint gate leaks none, at the price of keeping 44% of the non-sensitive objects local (Fig. 5b). Lowering the ratio threshold moves along the curve between these two points. Which operating point is appropriate depends on the application: the taint

gate protects better, but it removes much of the offloadable computation when the dependency graph is dense.

IX. SECURITY-AWARE REINFORCEMENT LEARNING

The reward function incorporates the vulnerability score so that the learning agents take security risk into account in addition to latency and energy. The secure reward of Eq. (6) subtracts $\sigma(V)$ whenever a task is offloaded; the higher the risk, the lower the payoff of offloading, which makes the agent more likely to execute sensitive tasks locally and to keep offloading low-risk, computationally heavy tasks when the network allows it. This is a soft constraint: the penalty makes unsafe offloading unattractive but does not forbid it. Whether the learned policies respect it is therefore a measurable question, which we answer in Section XI by probing the trained agents on high-vulnerability states, and the hard ODG gate is kept as a separate guarantee in front of the learned policy.

X. DEEP Q-NETWORK EXTENSION

Although Q-Learning performs effectively in small environments, its scalability is limited because state-action values must be stored explicitly in a lookup table, and a continuous state has to be discretised, which merges states that call for different actions. As the number of state variables increases, the Q-table grows exponentially. To overcome this limitation, the framework extends conventional Q-Learning with a Deep Q-Network (DQN): instead of storing explicit state-action values, a neural network is trained to approximate the action-value function [20], [21], [27].

The DQN receives five continuous inputs: battery level, task complexity, vulnerability score, available bandwidth (on a log scale) and edge-server load. The network has an input layer of five neurons, two hidden layers of 64 ReLU neu-

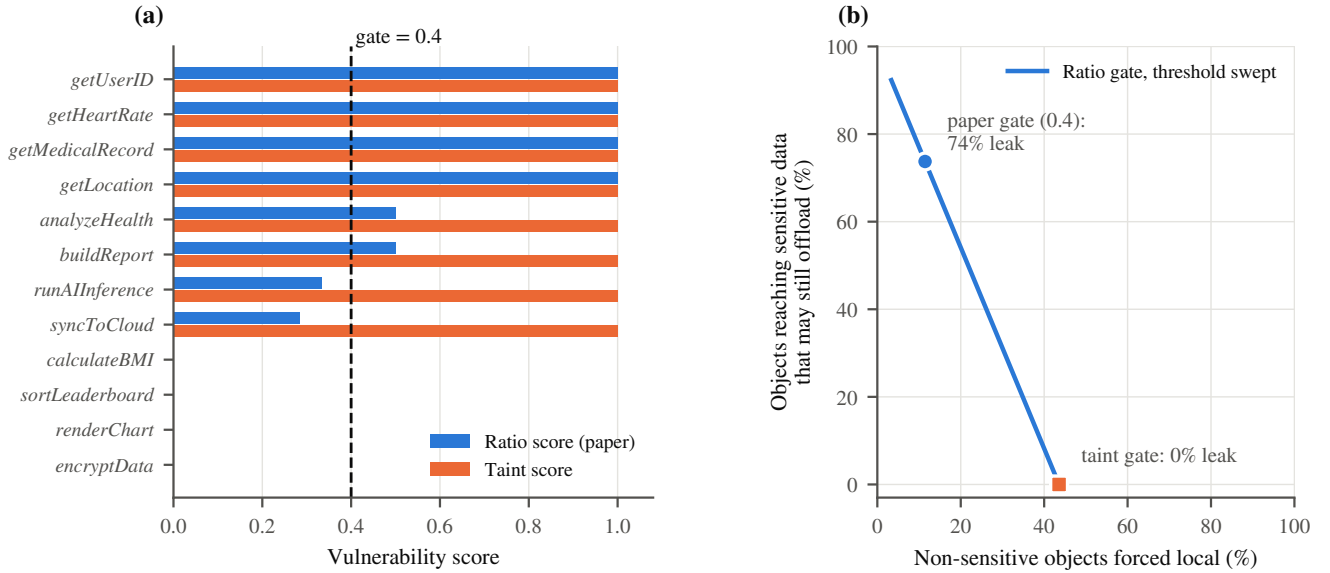


FIGURE 5. ODG vulnerability scoring. (a) Scores of the twelve objects of the example application under the ratio score of the paper (blue) and the conservative taint score (orange); the dashed line is the gate $\tau_V = 0.4$. Objects scoring above it must run locally. *syncToCloud* and *runAllInference* pass the ratio gate although they depend on sensitive objects. (b) On random dependency graphs, the percentage of objects that reach sensitive data but may still offload (vertical axis) against the percentage of non-sensitive objects forced local (horizontal axis) as the ratio threshold is swept (blue curve). The dot is the gate of the paper ($\tau_V = 0.4$); the square is the taint gate.

rons and an output layer of two actions, and approximates $Q(s, a; \theta)$ with parameters θ (4,674 weights in total). It is trained by minimizing the temporal-difference loss [13], [20]

$$\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2, \quad (13)$$

where θ^- are the parameters of a target network, with a clipped (Huber-style) gradient. Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer and mini-batches are sampled at random, which breaks the correlation between consecutive experiences; the target network is copied from the online network every 200 steps; and the exploration rate ϵ decays linearly from 1.0 to 0.05 over the first 60% of training. Table 2 gives the configuration.

XI. RESULTS AND DISCUSSION

The framework was evaluated through a series of experiments examining the bandwidth threshold, workload classification, queue control, vulnerability-aware offloading, and the learned decision policies. The bandwidth threshold, queue control and ODG results are reported in Sections III, VII and VIII; this section compares the decision policies end to end. All numbers are means over 10 seeds with 95% confidence intervals unless stated otherwise.

A. POLICY COMPARISON

Table 4 and Fig. 6 compare the policies on 200 fresh episodes per seed. Offloading everything is the worst choice: the edge load saturates and many tasks are sent over slow links, giving a reward of -4.43 per step against -2.07 for always executing locally. Offloading whenever the bandwidth exceeds R^*

improves on greedy offloading but still offloads about 46% of the tasks, including 46% of the high-vulnerability ones, and stays below local execution on reward (-2.36). Adding the ODG gate removes the security violations and raises the reward to -1.99 , and the load gate raises it slightly further to -1.97 for the complete rule pipeline.

The learned policies add to this. The DQN alone reaches -1.90 and the DQN behind the three gates -1.91 , compared with -1.97 for the rule pipeline, an improvement of about 0.06 reward per step (roughly 3%) that exceeds the 95% intervals, and a reduction of the latency per task from 0.88 s to 0.84 s. The learned policy offloads fewer tasks (14% against 26%), but chooses them better. Its reward is within the confidence interval of the myopic oracle (-1.89), which uses the true model; note that this oracle ignores the effect of a decision on later tasks and is therefore a reference, not an upper bound. The tabular agent behaves differently: used alone it reaches only -2.42 , below always-local execution, and it offloads 28% of the high-vulnerability tasks; behind the gates its reward rises to -1.99 , about the level of the threshold-plus-ODG rule. These gaps are modest. They are measured in a simulator whose rewards come from our own model of latency, energy and security, so they show that the learned policy exploits that model better than fixed rules, not that it would perform the same way on a real network.

B. LEARNING BEHAVIOUR

Fig. 7a shows the reward of the greedy policy on fresh episodes during training. The DQN passes the rule pipeline after about four thousand steps and then stays within the band between the rule pipeline and the oracle. The Q-table starts

TABLE 4. Comparison of offloading policies, mean $\pm$ 95% confidence interval over 10 seeds (200 evaluation episodes per seed). Reward, latency and energy are per task; “Offloaded” is the share of tasks sent to the edge; “High-vuln. offl.” is the share of tasks with vulnerability ≥ 0.66 that were offloaded; “Depleted” is the share of episodes in which the battery reached zero. “Gates” means the bandwidth, ODG and load gates of Fig. 2. The myopic oracle is a reference that uses the true model.

Policy	Reward/step	Latency (s)	Energy (J)	Offloaded (%)	High-vuln. offl. (%)	Depleted (%)
Always local	-2.07 ± 0.02	0.87 ± 0.00	0.78 ± 0.00	0	0.0	69
Always offload	-4.43 ± 0.14	1.78 ± 0.04	1.19 ± 0.06	100	100.0	75
Random	-3.43 ± 0.10	1.28 ± 0.03	1.13 ± 0.04	50	50.3	80
Threshold only	-2.36 ± 0.01	1.03 ± 0.00	0.64 ± 0.01	46	45.8	51
Threshold + ODG gate	-1.99 ± 0.02	0.90 ± 0.00	0.69 ± 0.01	29	0.0	57
Rule pipeline (3 gates)	-1.97 ± 0.02	0.88 ± 0.00	0.69 ± 0.01	26	0.0	56
Q-table	-2.42 ± 0.04	0.97 ± 0.01	0.81 ± 0.02	30	27.6	73
Q-table + gates	-1.99 ± 0.03	0.85 ± 0.00	0.74 ± 0.01	11	0.0	63
DQN	-1.90 ± 0.02	0.84 ± 0.00	0.70 ± 0.01	15	0.0	60
DQN + gates (full pipeline)	-1.91 ± 0.02	0.84 ± 0.01	0.71 ± 0.01	14	0.0	60
Myopic oracle	-1.89 ± 0.01	0.85 ± 0.00	0.68 ± 0.01	21	0.0	56

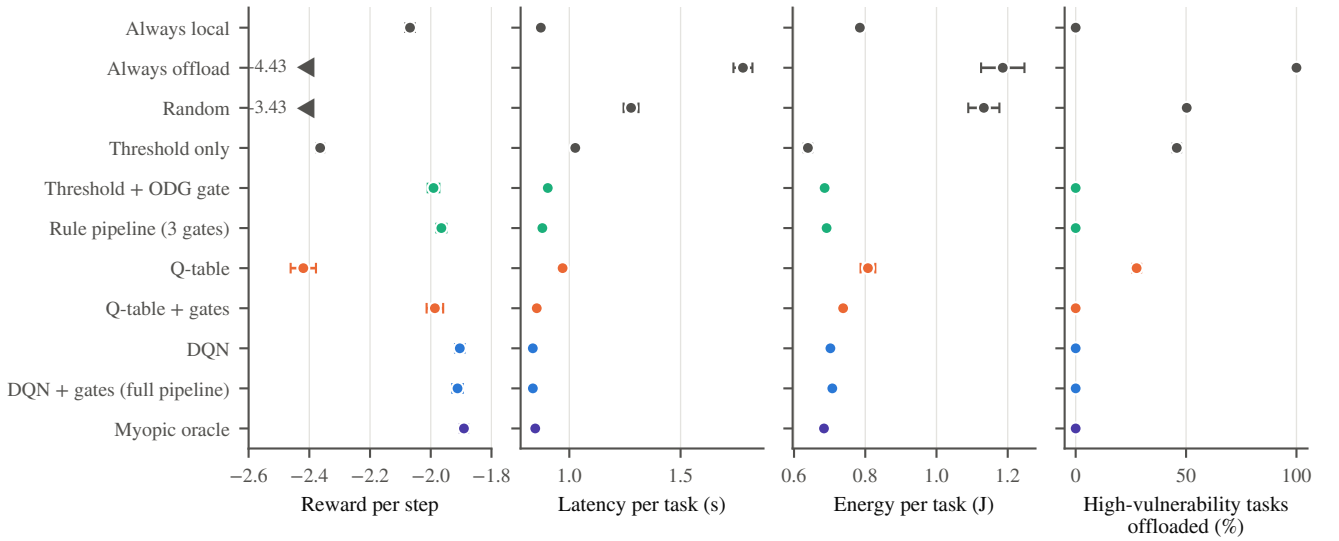


FIGURE 6. The policies of Table 4 on four metrics (dots: mean over 10 seeds; bars: 95% confidence interval, mostly smaller than the dots). Grey: simple baselines; green: rule-based gates; orange: Q-table; blue: DQN; violet: myopic oracle. Higher reward is better; lower latency, energy and high-vulnerability offloading are better. The two weakest policies in the reward panel lie outside the plotted range and are labelled with their values.

at the level of always-local (its table is initially zero), drops, recovers briefly and then deteriorates as training continues, ending below always-local. So the Q-table has not converged to a good policy at the discount factor of 0.95, and the sum of its Q-values would have been a misleading indicator of progress. Fig. 7b shows that the discount factor explains most of the difference: with $\gamma = 0$ the Q-table reaches -1.93 and is close to the DQN (-1.91), with $\gamma = 0.5$ it reaches -2.00 , and with $\gamma = 0.95$ only -2.40 , while the DQN stays at about -1.91 for all three values (five seeds each). In this environment nearly all of the reward of a decision is immediate; the only delayed effect is the edge load. Bootstrapping on a coarse, aliased discretisation of the state is a likely reason why a long horizon hurts the table, but we did not test this explanation. The practical conclusion is that the DQN is far less sensitive to this hyperparameter, and that the comparison with the Q-table is only fair if γ is tuned for it.

C. SECURITY OF THE LEARNED POLICIES

The learned agents receive the vulnerability score as an input and a penalty for offloading, but nothing forces them to respect it. We therefore probed the trained agents, without any gate, on 20,000 states visited by a random policy and report the share of high-vulnerability states ($V \geq 0.66$) in which they choose to offload (Table 5). The Q-table offloads 33.4% of them on average (up to 37.2% for some seed), whereas the DQN offloads none in any of the 10 seeds, including in the subset of high-vulnerability states in which the bandwidth, complexity, battery and load otherwise favour offloading. This is a property of the particular environment: the security penalty (3.0) exceeds the largest possible latency benefit, so a well-trained agent has no incentive to offload such tasks. It is an empirical result for these agents rather than a guarantee, since the penalty is a soft constraint, and a different penalty weight or training run could behave differently. The hard ODG gate in front of the learned policy is therefore kept: it

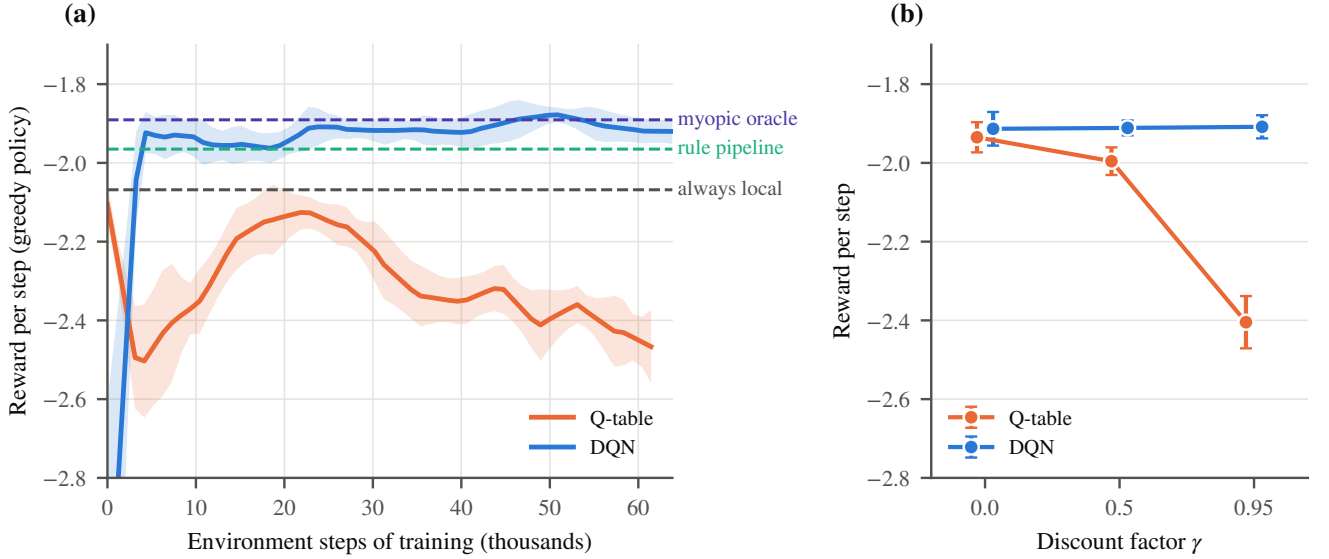


FIGURE 7. Learning behaviour. (a) Reward per step of the greedy policy on fresh episodes versus the number of training steps, for the Q-table (orange) and the DQN (blue); lines are means over 10 seeds and shaded bands the 95% confidence interval. Dashed lines are the always-local policy, the rule pipeline and the myopic oracle (evaluation episodes of Table 4). (b) Final reward per step of the two agents for three discount factors (five seeds; bars: 95% confidence interval).

TABLE 5. Share of states in which the trained agents choose to offload without any gate, on 20,000 states visited by a random policy (mean over 10 seeds; range over seeds in parentheses). High vulnerability means $V \geq 0.66$; “fav.” (favourable) additionally requires bandwidth above about 16 Mbps, complexity above 0.5, battery above 0.2 and edge load below 0.5.

Agent	All states (%)	High vuln. (%)	High vuln., fav. (%)
Q-table	40.1	33.4 (29.8–37.2)	34.4 (9.8–52.7)
DQN	16.4	0.0 (0.0–0.0)	0.0 (0.0–0.0)

enforces the property by construction.

D. WHAT THE DQN LEARNED

Fig. 8 shows the difference between the learned values of offloading and of local execution on three slices through the state space. In the first slice, the preference for offloading appears only at vulnerability below roughly 0.4 and at bandwidths above about 15–20 Mbps, close to the analytical thresholds of Eq. (3); at higher vulnerability, local execution is preferred even at 100 Mbps. The third slice shows that the bandwidth at which offloading becomes preferable rises as the edge load increases, which is the effect of the queueing delay in the reward. In the second slice, offloading is preferred for complex tasks at high battery levels, and the preference weakens at very low battery levels and for light tasks. The weaker preference at very low battery was not designed into the reward and we did not investigate its cause. These surfaces show that the network reproduces the structure that the model implies; since the reward is derived from our own cost model, they do not show that this structure is the right one for a real system.

E. SUMMARY

In the simulator, (i) the analytical threshold gives a clear first decision gate, (ii) a drift-plus-penalty admission controller bounds the edge queue and, at equal admission, collects more benefit than a fixed threshold, (iii) ODG gating keeps labelled sensitive objects on the device by construction, although the ratio score leaks transitively dependent objects, (iv) a DQN over the continuous state matches the myopic oracle and beats the rule pipeline by a small margin, and is more robust than a Q-table to the choice of discount factor, and (v) a hard gate in front of the learned policy is still advisable.

XII. COMPUTATIONAL COMPLEXITY

The analytical bandwidth threshold requires constant-time computation and therefore has complexity $O(1)$. The Q-Learning agent updates a single state-action pair at each interaction and its storage grows as $O(|S||A|)$ for $|S|$ states and $|A|$ actions; with the discretisation used here, $|S| = 1,440$, but $|S|$ grows multiplicatively with every additional variable or bin. The ODG vulnerability analysis requires traversal of object relationships; for a graph with V vertices and E edges, scoring one object costs $O(V+E)$. The admission controller evaluates one comparison per arriving task, which is $O(1)$ per task. For the DQN, forward inference is proportional to the number of weights, $O(W)$ with $W = 4,674$ here. Although DQN training adds computational overhead compared with tabular Q-Learning (in our runs about 45 seconds per seed against a few seconds for the table), it removes the discretisation and was less sensitive to the discount factor.

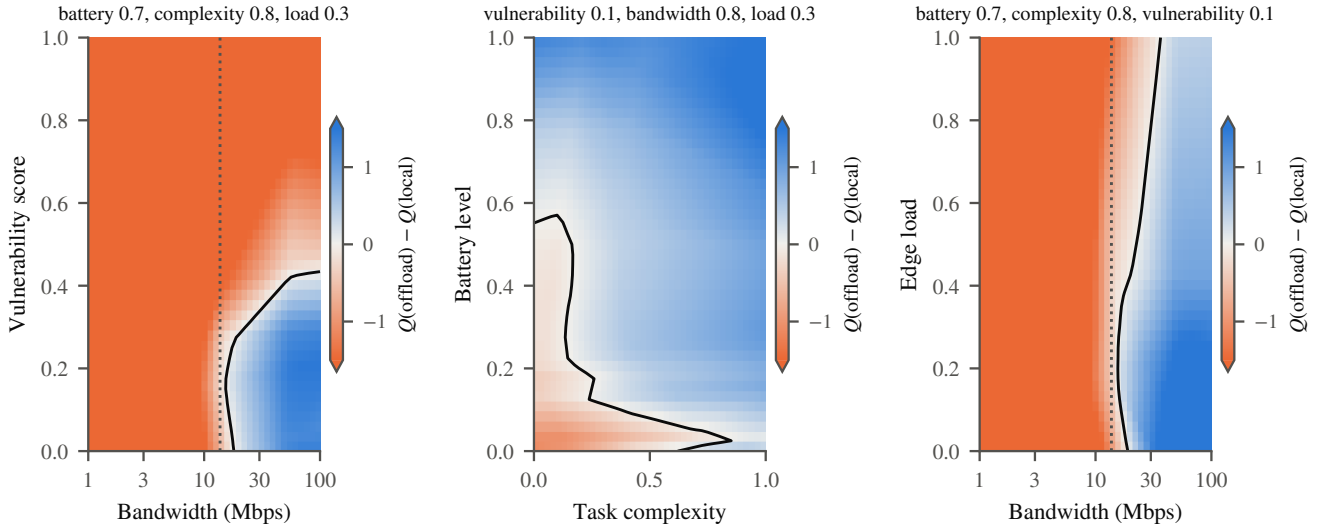


FIGURE 8. Learned DQN preference for offloading (seed 0) on three slices of the state space; colour shows $Q(\text{offload}) - Q(\text{local})$, clipped to ± 1.5 (blue: offloading preferred, orange: local execution preferred), and the black line is the decision boundary. (a) Bandwidth versus vulnerability score at battery 0.7, complexity 0.8, load 0.3. (b) Task complexity versus battery level at vulnerability 0.1, bandwidth 0.8 (normalised), load 0.3. (c) Bandwidth versus edge load at battery 0.7, complexity 0.8, vulnerability 0.1. The dotted vertical line marks the analytical threshold $R^* = 13.71$ Mbps.

XIII. LIMITATIONS

- **Simulation only.** There is no real-device or real-network evaluation. Latency, energy and security trade-offs come from our own cost model, with assumed power and battery values; the reward the agents optimise is defined by that model.
- **Workload proxy.** N-Queens stands in for CPU-bound tasks. Local times are measured on a desktop CPU and the edge time is modelled, not measured on a remote machine. In the policy experiments, tasks are abstract (complexity, data size) and do not run N-Queens.
- **Single edge server.** Multi-server orchestration and load balancing are not considered, and the edge load is a scalar state.
- **Learning problem.** Because the reward is derived from a known model and a decision affects later tasks mainly through the edge load, the learned policy largely matches a myopic optimum; the environment is only weakly sequential.
- **Modest margins.** The learned policy beats the rule pipeline by about 3% in reward; there is no comparison with algorithms from the literature or with real traces.
- **Queue controller.** The drift-plus-penalty rule and its simple bound are evaluated on Poisson arrivals with bounded benefits; per-user fairness and heterogeneous service times are not modelled.
- **ODG.** The example graph is hand-built, scores depend on how objects are labelled, and the ratio score can let objects with sensitive upstream dependencies offload. Random-graph results depend on the generator.
- **Soft security constraint.** The security behaviour of the learned agents is empirical; only the hard gate gives a guarantee.

- **Statistics.** 10 seeds for the main comparison and five for the discount-factor study; confidence intervals use a t distribution and assume approximate normality.

XIV. CONCLUSION

This paper studied a security-aware adaptive computation offloading framework for Mobile Edge Computing in simulation. The framework integrates an analytical bandwidth threshold, workload characterisation, reinforcement learning, Lyapunov drift-plus-penalty queue control, Object Dependency Graph vulnerability analysis, and Deep Q-Network-based continuous-state learning. The analytical model identified a break-even bandwidth of 13.71 Mbps (15.0 Mbps with the connection setup delay). The drift-plus-penalty controller bounded the simulated edge queue by $Vb_{\max} + A_{\max}$, and in the deterministic illustration the throttled queue stayed at 21 tasks against 297 for greedy admission. ODG gating kept labelled sensitive objects on the device, and a comparison with a stricter taint rule quantified how much the ratio score leaks. The DQN matched a myopic oracle and improved on the rule pipeline by a small margin; the Q-table was much more sensitive to the discount factor, and the learned agents respected the security penalty in our experiments but without a guarantee. The results illustrate the design under simplified assumptions and are not evidence of performance on real systems.

XV. FUTURE SCOPE

Several extensions can further improve the work: evaluation on real Android and iOS devices and edge hardware, with measured latency and energy; comparison against offloading algorithms from the literature and real workload and bandwidth traces; multi-server edge orchestration and load balanc-

ing; per-user admission control with fairness; runtime Object Dependency Graph generation from real applications; federated reinforcement learning for collaborative policy learning; energy-harvesting-aware offloading; and explainable reinforcement learning for security-critical applications. Settings such as healthcare monitoring, industrial IoT and smart-city analytics are natural candidates for such studies, but they were not evaluated here.

REFERENCES

- [1] K. Akherfi, M. Gerndt, and H. Harroud, "Mobile cloud computing for computation offloading: Issues and challenges," *Appl. Comput. Informat.*, vol. 14, no. 1, pp. 1–16, 2018.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [3] E. Cuervo *et al.*, "MAUI: Making smartphones last longer with code offload," in *Proc. MobiSys*, 2010, pp. 49–62.
- [4] B.-G. Chun *et al.*, "CloneCloud: Elastic execution between mobile device and cloud," in *Proc. EuroSys*, 2011, pp. 301–314.
- [5] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1584–1607, 2019.
- [6] J. Zhang, X. Hu, Z. Ning *et al.*, "Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2633–2645, 2018.
- [7] F. Wang, J. Xu, X. Wang, and S. Cui, "Joint offloading and computing optimization in wireless powered mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 1784–1797, 2018.
- [8] S. Barbarossa, S. Sardellitti, and P. D. Lorenzo, "Computation offloading for mobile cloud computing based on wide cross-layer optimization," in *Proc. Future Netw. Mobile Summit*, 2013.
- [9] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, 2016.
- [10] H. Zhu, C. Huang, and J. Yan, "Vulnerability evaluation for securely offloading mobile apps in the cloud," in *Proc. IEEE 2nd Int. Conf. Cloud Netw. (CloudNet)*, 2013.
- [11] S. Gan, M. Siew, C. Xu, and T. Q. S. Quek, "Differentially private deep Q-learning for pattern privacy preservation in MEC offloading," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 3578–3583.
- [12] N. Kiran, C. Pan, S. Wang, and C. Yin, "Joint resource allocation and computation offloading in mobile edge computing for SDN based wireless networks," *J. Commun. Netw.*, vol. 22, no. 1, pp. 1–11, 2020.
- [13] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, 2022.
- [14] L. Huang *et al.*, "Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing," *Digit. Commun. Netw.*, vol. 5, no. 1, pp. 10–17, 2019.
- [15] D. Kovachev and R. Klamka, "Framework for computation offloading in mobile cloud computing," *Int. J. Interact. Multimedia Artif. Intell.*, vol. 1, no. 7, pp. 6–15, 2012.
- [16] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network to minimize average task delay with deadline constraints," *Sensors*, vol. 24, no. 16, p. 5141, 2024.
- [17] S. Bi *et al.*, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7519–7537, 2021.
- [18] S. Bae, S. Han, and Y. Sung, "A reinforcement learning formulation of the Lyapunov optimization: Application to edge computing systems with queue stability," arXiv:2012.07279, 2020.
- [19] X. Zhao, G. Huang, J. Jiang, L. Gao, and M. Li, "Task offloading of cooperative intrusion detection system based on deep Q network in mobile edge computing," *Expert Syst. Appl.*, vol. 206, p. 117860, 2022.
- [20] C. Yang, X. Xu, X. Zhou, and L. Qi, "Deep Q network-driven task offloading for efficient multimedia data analysis in edge computing-assisted IoT," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 18, no. 2s, pp. 1–24, 2022.
- [21] J. Anand and B. Karthikeyan, "Adaptive and intelligent customized deep Q-network for energy-efficient task offloading in mobile edge computing environments," *Sci. Rep.*, vol. 16, p. 5456, 2026.
- [22] C. Liu, H. Wang, M. Zhao *et al.*, "Dependency-aware online task offloading based on deep reinforcement learning for IoT," *J. Cloud Comput.*, vol. 13, 2024.
- [23] J. Fang, D. Qu, H. Chen, and Y. Liu, "Dependency-aware dynamic task offloading based on deep reinforcement learning in mobile-edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1403–1415, 2024.
- [24] Y. Qin, J. Chen, L. Jin, R. Yao, and Z. Gong, "Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning," *Sci. Rep.*, vol. 15, p. 211, 2025.
- [25] J. Chen, L. Jin, R. Yao, and H. Zhang, "Deep reinforcement learning method for task offloading in mobile edge computing networks based on parallel exploration with asynchronous training," *Mobile Netw. Appl.*, vol. 30, no. 3–4, pp. 717–735, 2025.
- [26] X. Zhang, C. Fang, Z. Bai, L. Zhang, P. Wang, and Z. Cao, "Temporal dependency task offloading via deep reinforcement learning for mobile edge computing," *Peer-to-Peer Netw. Appl.*, vol. 18, no. 6, p. 288, 2025.
- [27] J. Sun, W. Zhang, M. Han *et al.*, "Deep reinforcement learning-based adaptive task offloading for mobile edge computing," *J. Supercomput.*, vol. 82, no. 5, 2026.
- [28] A. Jalal, U. Farooq, I. Rabbi *et al.*, "Towards intelligent edge computing through reinforcement learning based offloading in public edge as a service," *Sci. Rep.*, vol. 16, p. 4355, 2026.
- [29] J. Wang, J. Hu, G. Min *et al.*, "Dependent task offloading for edge computing based on deep reinforcement learning," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2449–2461, 2022.

...